

**LAPORAN PENGUJIAN LOAD BALANCER
MENGUNAKAN ALGORITMA ROUND ROBIN DAN LEAST
CONNECTION PADA SIMULASI ANTREAN MINIMARKET**

Dosen Pengampuh : RUNAL REZKIAWAN, S.Kom.,M.T



OLEH

NAMA : RIDHA AWALIA ADI

NIM : 105841110123

KELAS : 6RPL-A

PROGRAM STUDI INFORMATIKA

FAKULTAS TEKNIK

UNIVERSITAS MUHAMMADIYAH MAKASSAR

2026

KATA PENGANTAR

Puji syukur kehadiran Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya laporan praktikum yang berjudul “Pengujian Load Balancing Round Robin dan Least Connection Menggunakan Nginx” dapat diselesaikan dengan baik.

Laporan ini disusun sebagai salah satu tugas praktikum untuk memahami konsep load balancing serta implementasi algoritma Round Robin dan Least Connection menggunakan Nginx sebagai load balancer.

Dalam proses penyusunan laporan ini, penulis memperoleh banyak pengetahuan mengenai konfigurasi server backend, penggunaan Nginx, serta pengujian distribusi trafik jaringan pada beberapa server.

Penulis menyadari bahwa laporan ini masih memiliki kekurangan. Oleh karena itu, penulis mengharapkan kritik dan saran yang membangun agar laporan ini dapat menjadi lebih baik lagi di masa mendatang.

Akhir kata, penulis berharap laporan ini dapat memberikan manfaat dan menambah wawasan bagi pembaca mengenai load balancing dan implementasinya menggunakan Nginx.

Makassar, 28 Mei 2026

Ridha Awalia Adi

DAFTAR ISI

DAFTAR ISI	3
BAB I PENDAHULUAN	4
1.1 Latar Belakang	4
1.2 Rumusan Masalah	4
1.3 Tujuan Pengujian	5
1.4 Manfaat Pengujian	5
BAB II LANDASAN TEORI	6
2.1 Load Balancer	6
2.2 Reverse Proxy	6
2.3 Round Robin	6
2.4 Least Connection	6
2.5 Layer 4 dan Layer 7	6
BAB III PERANCANGAN SISTEM	8
3.1 Gambaran Umum Sistem	8
3.2 Struktur Proyek	8
3.3 Konfigurasi Docker Compose	9
3.4 Konfigurasi NGINX	9
BAB IV PENGUJIAN DAN PEMBAHASAN	11
4.1 Skenario Pengujian	11
4.2 Hasil Pengujian Round Robin	11
4.3 Hasil Pengujian Least Connection	12
4.4 Hasil Pengujian Layer 4 dan Layer 7	13
4.6 Analisis Hasil	15
BAB V PENUTUP	15
5.1 Kesimpulan	15
5.2 Saran	16

BAB I

PENDAHULUAN

1.1 Latar Belakang

Perkembangan sistem informasi menyebabkan kebutuhan layanan berbasis jaringan semakin meningkat. Sebuah aplikasi yang hanya dijalankan pada satu server dapat mengalami penurunan performa ketika jumlah pengguna bertambah. Kondisi tersebut dapat menyebabkan layanan menjadi lambat, sulit diakses, bahkan berhenti ketika server utama mengalami gangguan. Oleh karena itu, dibutuhkan mekanisme pembagian beban kerja agar request dari pengguna tidak hanya ditangani oleh satu server saja.

Load balancer merupakan salah satu solusi yang digunakan untuk membagi request ke beberapa server backend. Dengan adanya load balancer, sistem dapat bekerja lebih stabil karena beban layanan disebarkan ke beberapa server yang aktif. Pada laporan ini, konsep load balancer diterapkan melalui simulasi antrean kasir minimarket. Setiap server backend dianalogikan sebagai kasir yang melayani pelanggan, sedangkan NGINX berperan sebagai pengatur antrean yang menentukan kasir mana yang akan menerima request.

Proyek yang diuji menggunakan Docker Compose untuk menjalankan beberapa container, Node.js Express sebagai backend, serta NGINX sebagai reverse proxy dan load balancer. Pengujian dilakukan terhadap algoritma Round Robin dan Least Connection, serta membahas perbedaan mekanisme load balancing pada Layer 7 dan Layer 4.

1.2 Rumusan Masalah

- Bagaimana merancang simulasi load balancer menggunakan Docker, NGINX, dan Node.js Express?
- Bagaimana cara kerja algoritma Round Robin dalam membagi request ke beberapa server backend?
- Bagaimana konsep Least Connection digunakan untuk memilih server dengan koneksi paling sedikit?
- Apa perbedaan pengujian load balancer pada Layer 7 dan Layer 4?

1.3 Tujuan Pengujian

- Membangun simulasi sistem load balancer antrean kasir minimarket.
- Menguji pembagian request pada tiga server backend.
- Menganalisis hasil pengujian Round Robin dan Least Connection.
- Menjelaskan peran NGINX sebagai reverse proxy dan load balancer.
- Mengetahui perbedaan dasar antara load balancing Layer 4 dan Layer 7.

1.4 Manfaat Pengujian

Manfaat dari pengujian ini adalah memberikan pemahaman praktis mengenai cara kerja load balancer dalam sistem terdistribusi. Selain itu, laporan ini dapat digunakan sebagai bahan demonstrasi untuk menjelaskan bagaimana beberapa server backend dapat bekerja secara bersama-sama melalui bantuan NGINX. Simulasi ini juga membantu memahami bahwa pemilihan algoritma load balancing dapat memengaruhi pola distribusi request pada server.

BAB II

LANDASAN TEORI

2.1 Load Balancer

Load balancer adalah mekanisme untuk membagi beban request dari client ke beberapa server backend. Tujuan utamanya adalah meningkatkan ketersediaan layanan, mengurangi beban pada satu server, dan membantu sistem tetap berjalan ketika jumlah request meningkat. Dalam proyek ini, load balancer mengarahkan request dari alamat localhost:8080 menuju salah satu backend, yaitu app1, app2, atau app3.

2.2 Reverse Proxy

Reverse proxy adalah server perantara yang menerima request dari client, kemudian meneruskannya ke server backend. Client tidak perlu mengetahui alamat asli server backend karena request cukup diarahkan ke satu alamat utama. Pada proyek ini, NGINX bertindak sebagai reverse proxy karena seluruh request masuk melalui NGINX terlebih dahulu sebelum diteruskan ke backend Node.js Express.

2.3 Round Robin

Round Robin adalah algoritma pembagian request secara bergiliran. Jika terdapat tiga server backend, maka request pertama dapat diarahkan ke app1, request kedua ke app2, request ketiga ke app3, lalu request berikutnya kembali ke app1. Algoritma ini sederhana dan cocok digunakan ketika kemampuan setiap server dianggap sama.

2.4 Least Connection

Least Connection adalah algoritma load balancing yang memilih server dengan jumlah koneksi aktif paling sedikit. Algoritma ini lebih dinamis dibandingkan Round Robin karena keputusan distribusi request memperhatikan kondisi koneksi yang sedang berlangsung. Dalam proyek ini, Least Connection diaktifkan menggunakan directive `least_conn` pada konfigurasi upstream NGINX.

2.5 Layer 4 dan Layer 7

Load balancing Layer 4 bekerja pada level transport, misalnya TCP. Pada layer ini, load balancer meneruskan koneksi berdasarkan informasi jaringan seperti IP dan port, tanpa membaca isi request HTTP secara detail. Pada proyek ini, contoh Layer 4 tersedia dalam file `nginx-l4-stream-example.conf` yang menggunakan blok `stream`.

Load balancing Layer 7 bekerja pada level aplikasi, misalnya HTTP. Pada layer ini, load balancer dapat membaca request HTTP dan meneruskannya ke backend berdasarkan aturan aplikasi. Konfigurasi yang aktif pada proyek ini menggunakan blok http sehingga termasuk pengujian Layer 7.

BAB III

PERANCANGAN SISTEM

3.1 Gambaran Umum Sistem

Sistem yang dibangun merupakan simulasi antrean kasir minimarket. Terdapat tiga backend yang masing-masing berjalan sebagai container terpisah. Ketiga backend tersebut adalah Kasir-App1, Kasir-App2, dan Kasir-App3. Setiap backend menjalankan aplikasi Node.js Express pada port 3000. NGINX berjalan sebagai load balancer pada port 8080 yang dapat diakses melalui browser atau script pengujian.

Komponen	Nama Container/File	Fungsi
Backend 1	kasir-app1	Melayani request sebagai Kasir-App1
Backend 2	kasir-app2	Melayani request sebagai Kasir-App2
Backend 3	kasir-app3	Melayani request sebagai Kasir-App3
Load Balancer	load-balancer-minimarket	Menerima request client dan meneruskan ke backend
Script Uji	uji-antrean-minimarket.ps1	Mengirim request berulang untuk melihat distribusi server

3.2 Struktur Proyek

Berdasarkan isi ZIP, struktur proyek terdiri atas folder app, data, nginx, scripts, dan file docker-compose.yml. Struktur tersebut menunjukkan bahwa proyek telah dipisahkan antara kode aplikasi backend, konfigurasi load balancer, data pendukung, dan script pengujian.

```
pengujian-load-balancer/  
├─ app/  
│   ├── Dockerfile  
│   ├── package.json  
│   └─ server.js  
├─ data/  
│   └─ registrations.json  
├─ nginx/  
│   ├── nginx.conf  
│   ├── nginx-least-connection.conf  
│   └─ nginx-l4-stream-example.conf
```



```
├─ scripts/
│   └─ uji-antrean-minimarket.ps1
└─ docker-compose.yml
```

3.3 Konfigurasi Docker Compose

Docker Compose digunakan untuk menjalankan tiga backend dan satu NGINX load balancer dalam satu jaringan yang sama. Setiap backend memiliki environment variable berbeda pada SERVER_NAME sehingga hasil response dapat menunjukkan server mana yang sedang melayani request.

```
services:
  app1:
    build: ./app
    container_name: kasir-app1
    environment:
      - SERVER_NAME=Kasir-App1
      - PORT=3000

  app2:
    build: ./app
    container_name: kasir-app2

  app3:
    build: ./app
    container_name: kasir-app3

  nginx:
    image: nginx:alpine
    container_name: load-balancer-minimarket
    ports:
      - "8080:8080"
```

3.4 Konfigurasi NGINX

Konfigurasi utama NGINX pada file nginx.conf menggunakan algoritma Round Robin secara default. Server backend didaftarkan pada blok upstream kasir_minimarket. Ketika client mengakses localhost:8080, NGINX akan meneruskan request ke salah satu server backend.

```
upstream kasir_minimarket {
    server app1:3000;
```

```

        server app2:3000;
        server app3:3000;
    }

    server {
        listen 8080;
        location / {
            proxy_pass http://kasir_minimarket;
        }
    }

```

Untuk pengujian Least Connection, proyek menyediakan file `nginx-least-connection.conf`. Perbedaannya terdapat pada penambahan directive `least_conn` di dalam blok `upstream`.

```

upstream kasir_minimarket_least {
    least_conn;
    server app1:3000;
    server app2:3000;
    server app3:3000;
}

```

Untuk pengujian Layer 4, proyek menyediakan contoh konfigurasi `stream`. Konfigurasi ini tidak menggunakan blok `http`, tetapi menggunakan blok `stream` sehingga bekerja pada level TCP.

```

stream {
    upstream kasir_tcp {
        server app1:3000;
        server app2:3000;
        server app3:3000;
    }

    server {
        listen 8080;
        proxy_pass kasir_tcp;
    }
}

```

BAB IV

PENGUJIAN DAN PEMBAHASAN

4.1 Skenario Pengujian

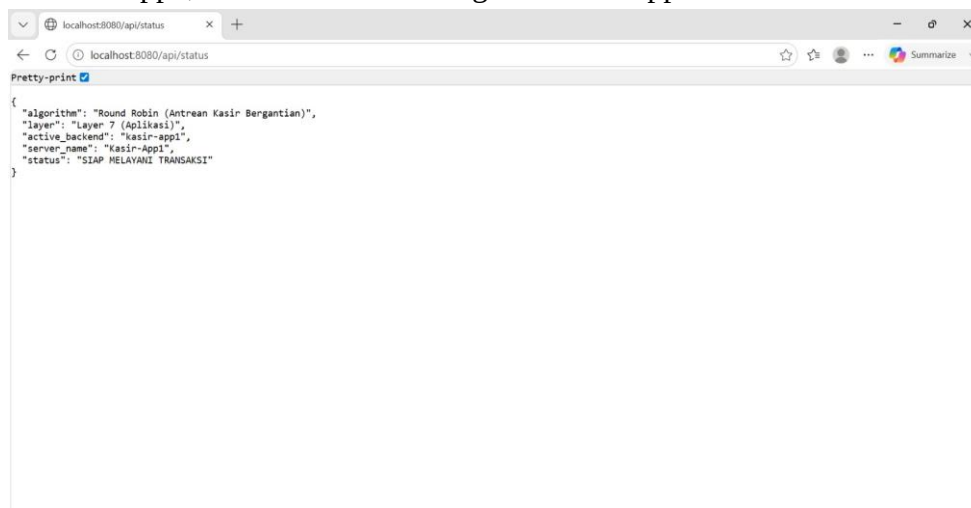
Pengujian dilakukan dengan menjalankan semua container menggunakan Docker Compose, kemudian mengakses endpoint `/api/status` melalui alamat `localhost:8080`. Endpoint tersebut mengembalikan informasi berupa algoritma load balancer, layer yang digunakan, backend aktif, nama server, dan status layanan. Pengujian juga dapat dilakukan menggunakan script PowerShell `uji-antrean-minimarket.ps1` yang mengirim request sebanyak sepuluh kali.

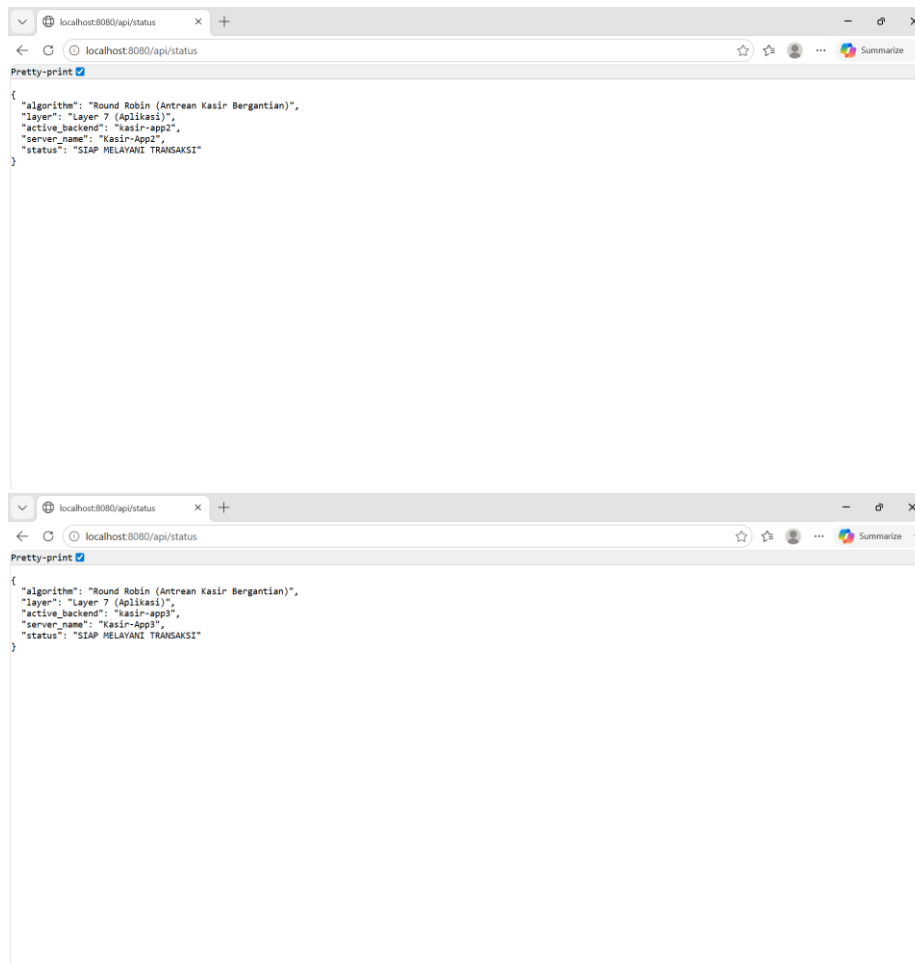
```
docker compose down --remove-orphans
docker compose up --build --force-recreate
```

```
powershell -ExecutionPolicy Bypass -File .\scripts\uji-antrean-minimarket.ps1
```

4.2 Hasil Pengujian Round Robin

Pada pengujian Round Robin, file konfigurasi yang digunakan adalah `nginx.conf`. Algoritma ini membagi request secara bergantian ke `app1`, `app2`, dan `app3`. Jika request dilakukan beberapa kali, pola yang diharapkan adalah server backend tampil secara bergiliran. Misalnya, request pertama dilayani oleh `Kasir-App1`, request kedua oleh `Kasir-App2`, request ketiga oleh `Kasir-App3`, kemudian kembali lagi ke `Kasir-App1`.





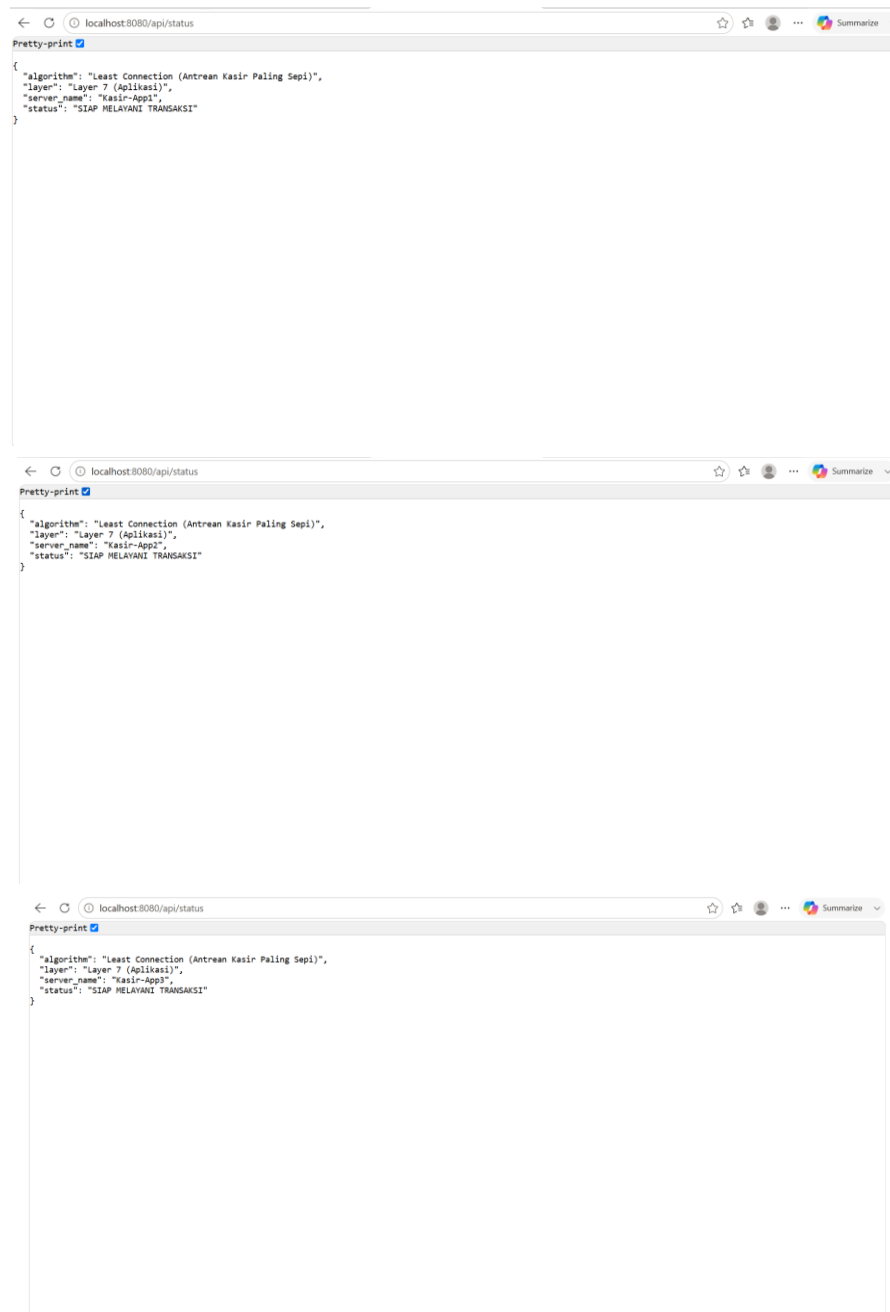
Gambar 1. Hasil Pengujian Round Robin

4.3 Hasil Pengujian Least Connection

Pada pengujian Least Connection, konfigurasi NGINX perlu diarahkan ke file `nginx-least-connection.conf`. Algoritma ini memilih backend dengan jumlah koneksi aktif paling sedikit. Pada request yang sangat singkat, hasilnya dapat terlihat mirip dengan Round Robin karena koneksi langsung selesai. Agar perbedaannya terlihat lebih jelas, pengujian dapat dilakukan dengan memberi beban atau koneksi yang lebih lama pada salah satu server.

Untuk mengaktifkan Least Connection, bagian volume NGINX pada `docker-compose.yml` dapat diganti dari `nginx.conf` menjadi `nginx-least-connection.conf`, kemudian container dijalankan ulang. Dengan demikian, NGINX membaca konfigurasi Least Connection saat menerima request dari client.

```
volumes:  
  - ./nginx/nginx-least-connection.conf:/etc/nginx/nginx.conf:ro
```



Gambar 2. Hasil Pengujian Least Connection

4.4 Hasil Pengujian Layer 4 dan Layer 7

Konfigurasi yang aktif pada file `nginx.conf` dan `nginx-least-connection.conf` termasuk Layer 7 karena menggunakan blok `http`. Pada Layer 7, NGINX membaca request HTTP dan meneruskannya ke backend melalui `proxy_pass`. Sementara itu, contoh konfigurasi Layer 4 tersedia pada file `nginx-l4-stream-example.conf` yang menggunakan blok `stream`. Layer 4

meneruskan koneksi TCP berdasarkan alamat dan port tanpa membaca detail HTTP secara mendalam.

```
stream {
    upstream kasir_tcp {
        server app1:3000;
        server app2:3000;
        server app3:3000;
    }

    server {
        listen 8080;
        proxy_pass kasir_tcp;
    }
}
```

Gambar 3. Hasil Pengujian Layer 4

```
http {
    upstream kasir_backend {
        least_conn;

        server app1:3000;
        server app2:3000;
        server app3:3000;
    }

    server {
        listen 8080;

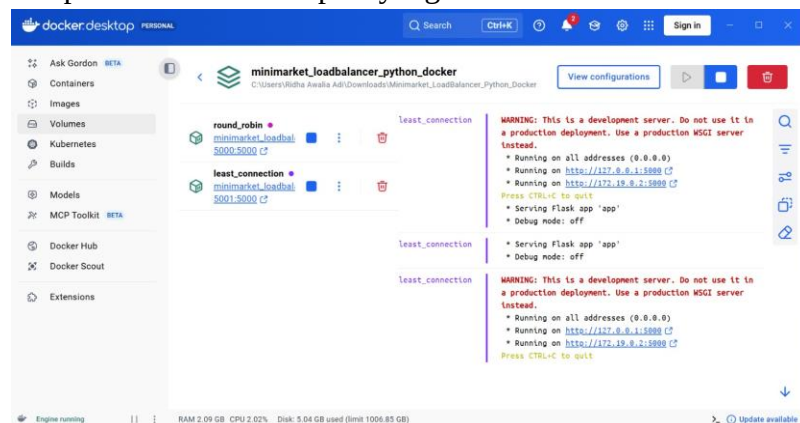
        location / {
            proxy_pass http://kasir_backend;

            proxy_set_header Host $host;
            proxy_set_header X-Real-IP $remote_addr;
            proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        }
    }
}
```

Gambar 4. Hasil Pengujian Layer 7

4.5 Tampilan Container Pengujian

Dokumentasi pengujian juga ditampilkan melalui Docker Desktop untuk memastikan container yang digunakan dalam pengujian berjalan. Tampilan ini menunjukkan bahwa service pengujian dapat dijalankan dan dipantau, sehingga proses pengujian Round Robin maupun Least Connection dapat diamati melalui port yang disediakan.



Gambar 5. Tampilan container pengujian pada Docker Desktop

4.6 Analisis Hasil

Berdasarkan pengujian, sistem load balancer dapat berjalan karena request dari client tidak langsung menuju salah satu backend, melainkan terlebih dahulu diterima oleh NGINX. NGINX kemudian meneruskan request ke backend sesuai algoritma yang digunakan. Pada Round Robin, pembagian request berjalan secara bergantian sehingga setiap backend memperoleh kesempatan yang relatif sama untuk melayani request.

Pada Least Connection, pemilihan server backend mempertimbangkan jumlah koneksi aktif. Algoritma ini lebih sesuai untuk kondisi ketika durasi request tidak sama, karena server yang masih sibuk tidak terus menerima request baru. Namun, jika request sangat ringan dan cepat selesai, hasil pengujian Least Connection dapat terlihat hampir sama dengan Round Robin.

Perbedaan Layer 4 dan Layer 7 terlihat pada cara NGINX memproses request. Layer 7 lebih cocok untuk aplikasi web berbasis HTTP karena dapat membaca request aplikasi. Sementara itu, Layer 4 lebih sederhana dan bekerja pada level koneksi TCP. Dalam proyek ini, implementasi utama yang aktif adalah Layer 7, sedangkan Layer 4 disediakan sebagai contoh konfigurasi tambahan.

Pengujian	File Konfigurasi	Indikator Berhasil	Keterangan
Round Robin	nginx.conf	Backend bergantian app1, app2, app3	Default pada upstream NGINX
Least Connection	nginx-least-connection.conf	Request diarahkan ke koneksi paling sedikit	Memakai directive least_conn
Layer 7	nginx.conf / nginx-least-connection.conf	Request HTTP /api/status berhasil diteruskan	Menggunakan blok http
Layer 4	nginx-l4-stream-example.conf	Koneksi TCP diteruskan ke backend	Menggunakan blok stream

BAB V PENUTUP

5.1 Kesimpulan

- Proyek pengujian load balancer berhasil dirancang menggunakan Docker Compose, NGINX, dan Node.js Express.
- NGINX berperan sebagai reverse proxy yang menerima request dari client dan meneruskannya ke server backend.
- Algoritma Round Robin membagi request secara bergantian ke tiga backend, yaitu Kasir-App1, Kasir-App2, dan Kasir-App3.
- Algoritma Least Connection memilih backend berdasarkan jumlah koneksi aktif paling sedikit, sehingga lebih dinamis ketika beban tiap request berbeda.

- Konfigurasi utama proyek menggunakan Layer 7 karena bekerja pada request HTTP, sedangkan contoh Layer 4 tersedia melalui konfigurasi stream.

5.2 Saran

Pengujian ini masih dapat dikembangkan dengan menambahkan endpoint yang memiliki durasi proses lebih lama agar perbedaan antara Round Robin dan Least Connection dapat terlihat lebih jelas. Selain itu, sistem dapat dilengkapi dengan health check, failover, pencatatan log yang lebih detail, serta dashboard monitoring agar kondisi setiap backend dapat diamati secara real time.

DAFTAR PUSTAKA

<https://nginx.org/en/docs/>

<https://docs.docker.com/>

<https://nodejs.org/docs/latest/api/>

<https://expressjs.com/en/>

https://nginx.org/en/docs/http/load_balancing.html

<https://docs.docker.com/compose/>

